EIN092B: Laboratorio de Interfaz gráfica para el procesamiento de imágenes
Guía de Actividades Sesión 12

4 de noviembre de 2026

1. Requisitos de entrada.

- Haber **entendido** y **realizado** las actividades de todas las sesiones anteriores.

2. Objetivos

- Comprender la arquitectura de aplicaciones de escritorio en Python utilizando la biblioteca estándar **Tkinter**.
- Diseñar e implementar una interfaz gráfica de usuario (GUI) nativa para la carga y manipulación interactiva de imágenes digitales.
- Aplicar filtros convolucionales (kernels) de suavizado y detección de bordes, conectando la lógica de *widgets* (botones, menús) a funciones de **OpenCV**.
- Implementar el Análisis de Componentes Principales (PCA) para la reconstrucción de imágenes, utilizando un *widget* de *slider* para la selección dinámica de componentes.

3. Indicaciones generales.

Las actividades planteadas en esta guía deben ser entregadas durante la sesión de laboratorio.

3.1. Sobre el proceso de revisión de actividades.

Para el proceso de revisión consideren lo siguiente:

- La revisión y evaluación de cada una de las actividades se realizará en dos etapas: (i) una validación básica de requerimientos funcionales mínimos durante la sesión, y (ii) una verificación de requerimientos funcionales y no funcionales posterior a la sesión.
- Durante la sesión, avise al staff (profesor o ayudante) cuando complete una o varias actividades para que muestre y explique sus resultados, y responda las preguntas que se le realizarán sobre los conceptos asociados.

- Cualquier integrante del grupo debe ser capaz de responder las preguntas. Si las respuestas no son satisfactorias, o algún integrante no es capaz de responder, no se revisará la actividad hasta que demuestren que TODOS los integrantes del grupo demuestren un entendimiento mínimo sobre lo que están entregando.
- Durante la sesión, el staff marcará en una planilla cuando Ud. considere una actividad terminada y haya pasado la interrogación. El tener la actividad marcada como revisada en la sesión no entrega puntaje, y solo asegura que cumple con requerimientos para que entregue sus archivos y pase a la siguiente etapa de revisión. El formato de entrega de sus archivos se indica en la Sección 3.2.
- En el caso de las actividades no entregadas en la sección correspondiente, contará como no entregada a tiempo y **deberá entregar la actividad corregida en la siguiente sesión, aplicando descuentos por atraso (penalización correspondiente al 35 % del puntaje inicial si se entrega en la siguiente sesión. Si se entrega en dos sesiones posteriores, 50 %**).
- En el caso de que algún integrante del grupo no este presente cuando su grupo correspondiente presente la evaluación y su correspondiente interrogación, deberá presentarlo por su propia cuenta en las siguientes sesiones y sufriendo la penalización por atraso.

3.2. Indicaciones de formato para entrega.

La evaluación constará de dos partes. La primera parte consiste en una validación básica y preguntas sobre el código propuesto por el grupo. La segunda parte consistirá en una verificación del código y del informe en PDF, la cual se realizará sobre los archivos que deberá entregar vía Aula USM, los cuales deben seguir estrictamente las especificaciones que se indican a continuación:

1. Debe crear una carpeta con el nombre `GXX_Sesion12`, donde XX es el número de grupo. Por ejemplo, la carpeta para el grupo 5 se llamaría `G5_Sesion12`. El número de su grupo fue definido en la sesión anterior.
2. Dentro de esta carpeta, deben incluirse únicamente los siguientes dos archivos:
   a) **Archivo de código (`.py`):** Un único script de Python con la aplicación completa, que pueda ejecutarse sin modificaciones.
   b) **Informe de uso (`.pdf`):** Un documento simple que muestre el funcionamiento de la aplicación, con una captura de pantalla de la ventana por cada acción realizada:
      - Imagen original cargada.
      - Filtro de suavizado (Blur).
      - Filtro de bordes (Laplace).
      - Reconstrucción con PCA para al menos tres valores de k.

      Cada captura debe incluir un breve epígrafe que indique la acción y, en el caso de PCA, el valor de k utilizado. Se recomienda usar la misma imagen en todas las capturas para poder comparar los resultados.

Recordar que **será responsabilidad del grupo entregar de manera correcta los archivos**. Se permitirá el envío de los archivos hasta las **23:59 del 15 de Noviembre**. Sin embargo, solo se evaluarán las actividades marcadas como revisadas durante la sesión. Cualquier actividad adicional no será considerada en la revisión.

4. Introducción

En lugar de depender de aplicaciones web, podemos construir interfaces nativas de escritorio que se ejecutan directamente en el sistema operativo. La biblioteca estándar de Python para esto es **Tkinter**.

4.1. Tkinter: Interfaces Nativas en Python

Tkinter ("Tk interface") es el kit de herramientas de GUI estándar de Python. Es una envoltura ligera alrededor de Tcl/Tk, un lenguaje de scripting para GUIs. Al estar en la biblioteca estándar, no requiere instalación.

Conceptos Clave de Tkinter:

1. **Ventana Raíz (Root Window):** Es la ventana principal de la aplicación, creada con `tk.Tk()`.
2. **Widgets:** Son los componentes de la GUI (botones, etiquetas, sliders, etc.). Ejemplos: `tk.Button`, `tk.Label`, `tk.Scale`.
3. **Gestores de Geometría:** Son los métodos que organizan los *widgets* en la ventana. Los más comunes son `.pack()` (apila widgets) y `.grid()` (los organiza en una grilla).
4. **Manejo de Eventos:** Se conecta la interacción del usuario (ej. clic de botón) a una función de Python usando el argumento `command`.
5. **Loop Principal (Main Loop):** `root.mainloop()` es una llamada bloqueante que inicia el bucle de eventos de la aplicación, esperando la interacción del usuario.

```python
import tkinter as tk

# 1. Crear la ventana raiz
root = tk.Tk()
root.title("Mi Primera App")

# 2. Crear un widget
# tk.Label es una etiqueta de texto
label = tk.Label(root, text="Hola, mundo!")

# 3. Empaquetar el widget
# .pack() lo coloca en la ventana
label.pack(pady=10, padx=10)

# 4. Iniciar el bucle de eventos
root.mainloop()
```

Tkinter y OpenCV/NumPy: Tkinter no puede mostrar una imagen de NumPy (OpenCV) directamente. Necesitamos una biblioteca intermediaria: **Pillow** (PIL). El flujo es: **NumPy (OpenCV)** $\rightarrow$ **PIL Image** $\rightarrow$ **ImageTk.PhotoImage** $\rightarrow$ **tk.Label**

4.2. Análisis de Componentes Principales (PCA) en Imágenes

El **Análisis de Componentes Principales (PCA)** es una técnica de reducción de dimensionalidad. Su objetivo es transformar un conjunto de datos con variables posiblemente correlacionadas en un nuevo conjunto de variables linealmente no correlacionadas llamadas **componentes principales**.

Estas componentes se ordenan según la cantidad de **varianza** original que "explican". La primera componente principal captura la mayor cantidad de varianza, la segunda la siguiente, y así sucesivamente.

PCA para Reconstrucción de Imágenes: Una imagen en escala de grises de $N \times M$ píxeles puede ser tratada como una matriz. Al aplicar PCA, podemos "comprimir"la imagen reteniendo solo las primeras k componentes principales.

La reconstrucción de la imagen se realiza utilizando la transformación inversa de PCA, pero usando solo el subconjunto k de componentes.

$$X_{reconstruido} = \text{Transformada}_k \cdot \text{Componentes}_k^T + \mu,$$

donde μ es la media de los datos originales.

Implementación con Scikit-Learn: El código de `scikit-learn` para calcular el PCA es independiente de la GUI y se puede reutilizar.

```python
import numpy as np
from sklearn.decomposition import PCA

def aplicar_pca(imagen_np, k):
    # 1. Normalizar (Centrar la media)
    img_norm = imagen_np - np.mean(imagen_np, axis=0)

    # 2. Aplicar PCA
    pca = PCA(n_components=k)
    transformed = pca.fit_transform(img_norm)

    # 3. Reconstruir
    reconstructed = pca.inverse_transform(transformed)

    # 4. Devolver a la escala original (sumar la media)
    reconstructed_img = reconstructed + np.mean(imagen_np, axis=0)

    # 5. Asegurar que los valores estan en el rango de pixeles
    reconstructed_img = np.clip(reconstructed_img, 0, 255).astype(np.uint8)

    return reconstructed_img
```

5. Actividades

El objetivo es construir una aplicación en una sola clase de Python. Usaremos una estructura Orientada a Objetos, que es la forma estándar de construir aplicaciones en Tkinter.

Estructura Base (Esqueleto de la App): Copie este código como su punto de partida.

```python
import tkinter as tk
from tkinter import filedialog
from PIL import Image, ImageTk
import cv2
import numpy as np
from sklearn.decomposition import PCA

class ImageProcessorApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Procesador de Imagenes")

        # Guardar referencias a las imagenes
        self.original_image_np = None # Imagen en formato NumPy
        self.processed_image_tk = None # Imagen en formato Tkinter

        # --- Configurar la GUI ---

        # 1. Frame de controles
        control_frame = tk.Frame(self.root)
        control_frame.pack(side=tk.TOP, fill=tk.X, padx=10, pady=5)

        self.load_button = tk.Button(control_frame, text="Cargar Imagen", command
    =self.cargar_imagen)
        self.load_button.pack(side=tk.LEFT)

        # 2. Frame de visualizacion (dividido en dos)
        display_frame = tk.Frame(self.root)
        display_frame.pack(fill=tk.BOTH, expand=True)

        self.original_label = tk.Label(display_frame, text="Imagen Original")
        self.original_label.pack(side=tk.LEFT, fill=tk.BOTH, expand=True, padx=5)

        self.processed_label = tk.Label(display_frame, text="Imagen Procesada")
        self.processed_label.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True, padx
    =5)

    def cargar_imagen(self):
        # A implementar en Actividad 1
        pass

    def aplicar_filtro(self):
        # A implementar en Actividad 2
        pass

    def reconstruir_pca(self):
        # A implementar en Actividad 3
        pass

```

```python
# --- Funcion para convertir de NumPy (OpenCV) a Tkinter Image ---
def np_to_tkimage(np_image):
    # OpenCV usa BGR, PIL usa RGB (si es color)
    # Para escala de grises, es mas simple
    if len(np_image.shape) == 2: # Escala de grises
        img_pil = Image.fromarray(np_image)
    else: # Color
        img_pil = Image.fromarray(cv2.cvtColor(np_image, cv2.COLOR_BGR2RGB))

    return ImageTk.PhotoImage(image=img_pil)


if __name__ == "__main__":
    root = tk.Tk()
    root.geometry("800x600") # Tamano inicial
    app = ImageProcessorApp(root)
    root.mainloop()
```

5.1. Actividad 1: Carga y Visualización de Imagen

Complete el método `cargar_imagen`.

Lineamientos:

- Use `filedialog.askopenfilename()` para mostrar un diálogo de selección de archivo.
- Lea la imagen usando `cv2.imread()` en modo escala de grises (`cv2.IMREAD_GRAYSCALE`).
- Almacene esta imagen de NumPy en `self.original_image_np`.
- Convierta la imagen de NumPy a una imagen de Tkinter usando la función `np_to_tkimage` proporcionada.
- **Importante:** Guarde la referencia de la imagen de Tkinter en un atributo de la clase (ej. `self.original_image_tk`). Si la guarda en una variable local, Python la borrará (garbage collector) y la imagen no se mostrará.
- Configure la `self.original_label` para que muestre esta imagen.

```python
    def cargar_imagen(self):
        # 1. Pedir al usuario el path del archivo
        file_path = filedialog.askopenfilename()
        if not file_path:
            return # El usuario cancelo

        # 2. Leer la imagen con OpenCV en escala de grises
        self.original_image_np = cv2.imread(file_path, cv2.IMREAD_GRAYSCALE)

        # 3. Convertir a formato Tkinter
        tk_image = np_to_tkimage(self.original_image_np)

```

```
        # 4. Mostrar en la etiqueta original
        self.original_label.config(image=tk_image, text="")

        # 5. GUARDAR LA REFERENCIA (CRUCIAL)
        self.original_image_tk = tk_image

        # 6. Tambien mostrarla en el panel de procesado al inicio
        self.processed_label.config(image=tk_image, text="")
        self.processed_image_tk = tk_image
```

5.2. Actividad 2: Aplicación de Kernels Interactivos

Agregue un menú de selección (dropdown) y un botón para aplicar filtros.

Lineamientos:

- Modifique el `__init__` para añadir un `tk.StringVar` y un `tk.OptionMenu` al `control_frame`.
- Las opciones deben ser: 'Original', 'Suavizado (Blur)', 'Bordes (Laplace)'.
- Añada un `tk.Button` "Aplicar Filtro" que llame a `self.aplicar_filtro`.
- Complete el método `aplicar_filtro`:
- Verifique que se haya cargado una imagen (`self.original_image_np is not None`).
- Obtenga la selección del `tk.StringVar`.
- Aplique el kernel correspondiente usando `cv2.filter2D`.
- Muestre la imagen resultante en `self.processed_label` (recordando el flujo NumPy → PIL → ImageTk y guardando la referencia).

```
        # ... (despues del boton de cargar)

        # Menu de seleccion de filtro
        self.filter_options = ['Original', 'Suavizado (Blur)', 'Bordes (Laplace)']
        self.filter_var = tk.StringVar(self.root)
        self.filter_var.set(self.filter_options[0]) # Valor por defecto

        filter_menu = tk.OptionMenu(control_frame, self.filter_var, *self.filter_options)
        filter_menu.pack(side=tk.LEFT, padx=5)

        apply_filter_btn = tk.Button(control_frame, text="Aplicar Filtro", command=self.aplicar_filtro)
        apply_filter_btn.pack(side=tk.LEFT)
```

```python
    def aplicar_filtro(self):
        if self.original_image_np is None:
            return # No hay imagen cargada

        filtro = self.filter_var.get()
        processed_np = None

        if filtro == 'Original':
            processed_np = self.original_image_np

        elif filtro == 'Suavizado (Blur)':
            kernel_blur = np.ones((5, 5), np.float32) / 25.0
            processed_np = cv2.filter2D(self.original_image_np, -1, kernel_blur)

        elif filtro == 'Bordes (Laplace)':
            # SU CODIGO AQUI: Defina el kernel Laplaciano (ej. 3x3)
            kernel_laplace = np.array([[0, 1, 0], [1, -4, 1], [0, 1, 0]], np.
    float32)
            # SU CODIGO AQUI: Aplique el filtro
            processed_np = cv2.filter2D(self.original_image_np, -1,
    kernel_laplace)

        # Mostrar la imagen procesada
        tk_image = np_to_tkimage(processed_np)
        self.processed_label.config(image=tk_image)
        self.processed_image_tk = tk_image # Guardar referencia
```

5.3. Actividad 3: PCA y Reconstrucción Interactiva

Agregue un *slider* (llamado `tk.Scale`) para controlar el número de componentes de PCA.

Lineamientos:

- Modifique el `__init__` para añadir un `tk.Scale` y un botón Reconstruir PCA".
- El `tk.Scale` debe ser horizontal (`orient=tk.HORIZONTAL`).
- El rango del `tk.Scale` es un desafío: no sabemos el tamaño de la imagen al inicio. Configúrelo de 1 a 100 por ahora. Lo ideal es actualizar su rango (opción `to=...`) después de cargar la imagen.
- Complete el método `reconstruir_pca`:
- Obtenga el valor del slider (`k = self.pca_slider.get()`).
- Llame a la función `aplicar_pca` (definida en la Introducción, debe copiarla en su script) con `self.original_image_np` y `k`.
- Muestre la imagen reconstruida en `self.processed_label`.

```
        # ... (dentro de control_frame)

        # Slider para PCA
        self.pca_slider = tk.Scale(control_frame, from_=1, to=100, orient=tk.
    HORIZONTAL, label="Componentes PCA")
        self.pca_slider.pack(side=tk.LEFT, padx=10)

        apply_pca_btn = tk.Button(control_frame, text="Reconstruir PCA", command=
    self.reconstruir_pca)
        apply_pca_btn.pack(side=tk.LEFT)
```

```
    def reconstruir_pca(self):
        if self.original_image_np is None:
            return

        # Obtener k del slider
        k = int(self.pca_slider.get())

        # (Mejora opcional: actualizar el 'to' del slider en cargar_imagen)
        # max_comp = min(self.original_image_np.shape)
        # self.pca_slider.config(to=max_comp)

        # SU CODIGO AQUI: Llame a la funcion de PCA
        # (Asegurese de haber copiado la funcion aplicar_pca en su script)
        reconstructed_np = aplicar_pca(self.original_image_np, k)

        # Mostrar resultado
        tk_image = np_to_tkimage(reconstructed_np)
        self.processed_label.config(image=tk_image)
        self.processed_image_tk = tk_image # Guardar referencia
```

Referencias

[1] Python Software Foundation. *tkinter — Python interface to Tcl/Tk*. https://docs.python.org/3/library/tkinter.html

[2] Scikit-learn: Machine Learning in Python. *sklearn.decomposition.PCA*. https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html

[3] OpenCV (2025). *OpenCV Python Tutorials*. https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html